

LAPORAN PRAKTIKUM MOBILE PROGRAMMING

MODUL 9

(StatefulWidget)



Nama : Dimas Bagus Hari Murti

NIM : 240605110080

Kelas : B

Tanggal : 8 September 2025

JURUSAN TEKNIK INFORMATIKA

FAKULTAS SAINS DAN TEKNOLOGI

UNIVERSITAS ISLAM NEGERI MAULANA MALIK IBRAHIM

MALANG

GANJIL 2025/2026

I. Tujuan

Tujuan dari percobaan ini adalah untuk mempelajari penerapan `StatefulWidget` pada Flutter sebagai komponen yang mampu mengubah tampilan aplikasi secara dinamis. Melalui praktikum ini, praktikan diharapkan dapat memahami perbedaan mendasar antara `StatelessWidget` yang bersifat statis dan `StatefulWidget` yang menyimpan serta memperbarui state selama aplikasi berjalan. Selain itu, praktikum ini juga bertujuan untuk melatih penggunaan fungsi `setState()` sebagai metode pembaruan state agar perubahan data dapat langsung ditampilkan pada UI. Dengan menerapkan contoh sederhana seperti Counter App dan Like Button, praktikan dapat melihat bagaimana nilai state dapat meningkat, berkurang, atau berubah status berdasarkan interaksi pengguna. Secara keseluruhan, melalui kegiatan ini praktikan memperoleh dasar pemahaman mengenai pengelolaan state dalam Flutter, yang nantinya menjadi fondasi penting sebelum mempelajari manajemen state yang lebih kompleks dengan menggunakan metode seperti Provider, BLoC, dan Riverpod.

II. Langkah Kerja

- Membuat proyek Flutter baru dengan nama `counter_app` sebagai proyek utama untuk penerapan `StatefulWidget`.
- Membuka file `main.dart`, kemudian menghapus kode bawaan hingga hanya menyisakan bagian `import` dan fungsi `main()`.
- Membuat kelas `MyApp` yang menggunakan `StatelessWidget` sebagai struktur awal aplikasi dan mengarahkan halaman utama ke `CounterPage()`.
- Menambahkan class `CounterPage` sebagai `StatefulWidget`, kemudian membuat class `state _CounterPageState` sebagai tempat penyimpanan dan pengelolaan *state*.
- Mendeklarasikan variabel `_counter` yang digunakan sebagai nilai hitungan yang akan ditampilkan pada tampilan aplikasi.

- Menambahkan fungsi `_incrementCounter()` dan `_decrementCounter()` untuk menambah dan mengurangi nilai `_counter` menggunakan `setState()` agar UI berubah secara real-time.
- Mengatur tampilan pada method `build()` dengan menampilkan angka counter serta dua tombol untuk mengurangi dan menambah nilai.
- Menjalankan aplikasi dan memastikan perubahan angka terjadi ketika tombol ditekan.

III. Screenshot Hasil

1. Percobaan 1 – Counter App

Source code:

```

import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      title: 'Stateful Widget Demo',
      home: CounterPage(),
    );
  }
}

class CounterPage extends StatefulWidget {
  const CounterPage({super.key});

  @override
  _CounterPageState createState() => _CounterPageState();
}

class _CounterPageState extends State<CounterPage> {
  int _counter = 0; // state

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Counter App')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Text(
              'Klik tombol tambah atau kurang:',
              style: TextStyle(fontSize: 24),
            ),
            Text('$_counter', style: const TextStyle(fontSize: 40)),
            Row(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                ElevatedButton(
                  style: ElevatedButton.styleFrom(
                    backgroundColor: Colors.blue[700]!,
                    minimumSize: const Size(100, 60),
                  ),
                  onPressed: _decrementCounter,
                  child: const Text(
                    'Kurang',
                    style: TextStyle(color: Colors.white, fontSize: 24),
                  ),
                ),
                ElevatedButton(
                  style: ElevatedButton.styleFrom(
                    backgroundColor: Colors.blue[700]!,
                    minimumSize: const Size(100, 60),
                  ),
                  onPressed: _incrementCounter,
                  child: const Text(
                    'Tambah',
                    style: TextStyle(color: Colors.white, fontSize: 24),
                  ),
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}

```

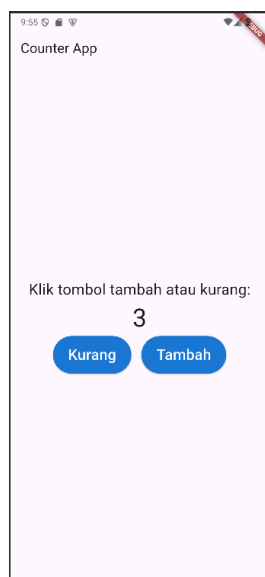
```

75
76     void _incrementCounter() {
77         setState() {
78             _counter++;
79         };
80     }
81
82     void _decrementCounter() {
83         setState() {
84             _counter--;
85         };
86     }
87 }

```

Program Counter App ini merupakan contoh penerapan StatefulWidget pada Flutter dimana nilai `_counter` dapat berubah sesuai interaksi pengguna. Perubahan nilai dilakukan melalui dua fungsi, yaitu `_incrementCounter()` untuk menambah dan `_decrementCounter()` untuk mengurangi nilai counter. Kedua fungsi tersebut memanggil `setState()` sehingga tampilan UI diperbarui secara langsung tanpa perlu memuat ulang aplikasi. Antarmuka program terdiri dari teks instruksi, tampilan angka, serta dua tombol yang disusun menggunakan Column dan Row. Ketika aplikasi dijalankan, angka awal ditampilkan sebagai 0, lalu berubah meningkat atau menurun saat tombol Tambah atau Kurang ditekan. Output ini menunjukkan bahwa pengelolaan state berjalan dengan baik dan aplikasi mampu merespon input pengguna secara real-time.

Output:



2. Percobaan 2 – Like button

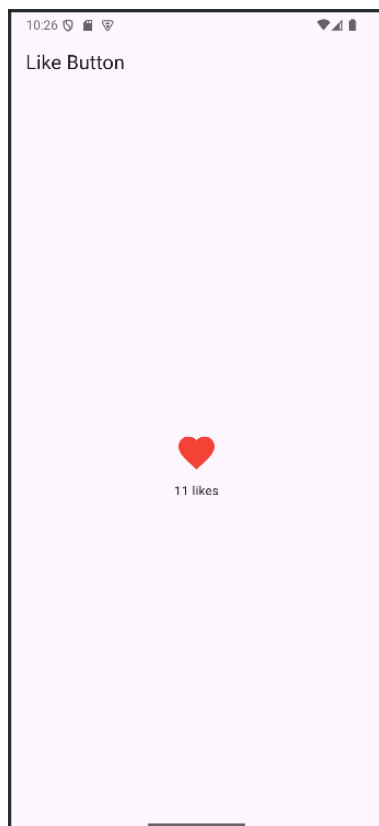
```
21 class LikeButtonPage extends StatefulWidget {  
22   const LikeButtonPage({super.key});  
23  
24   @override  
25   State<LikeButtonPage> createState() => _LikeButtonPageState();  
26 }  
27  
28 class _LikeButtonPageState extends State<LikeButtonPage> {  
29   bool _isLiked = false; // state untuk status like  
30   int _likeCount = 10;  
31  
32   @override  
33   Widget build(BuildContext context) {  
34     return Scaffold(  
35       appBar: AppBar(title: const Text('Like Button')),  
36       body: Center(  
37         child: Column(  
38           mainAxisAlignment: MainAxisAlignment.center,  
39           children: [  
40             IconButton(  
41               icon: Icon(  
42                 _isLiked ? Icons.favorite : Icons.favorite_border,  
43                 color: _isLiked ? Colors.red : Colors.grey,  
44                 size: 48,  
45               ),  
46               onPressed: _toggleLike,  
47             ),  
48             Text('${_likeCount} likes'),  
49           ],  
50         ),  
51       ),  
52     );  
53   }  
54 }
```

```
54 void _toggleLike() {  
55   setState(() {  
56     if (_isLiked) {  
57       _likeCount--;  
58       _isLiked = false;  
59     } else {  
60       _likeCount++;  
61       _isLiked = true;  
62     }  
63   });  
64 }  
65 }
```

Program Like Button ini menunjukkan penggunaan StatefulWidget untuk mengelola perubahan state pada ikon like dan jumlah suka secara dinamis. Variabel `_isLiked` digunakan sebagai penanda apakah tombol sedang dalam kondisi suka atau tidak, sedangkan `_likeCount` menyimpan jumlah like yang akan bertambah atau berkurang berdasarkan interaksi pengguna. Ketika ikon ditekan, fungsi `_toggleLike()` dipanggil dan `setState()` digunakan agar perubahan data langsung diperbarui pada tampilan UI. Jika ikon sebelumnya sudah dalam keadaan like, maka jumlah like dikurangi dan ikon berubah menjadi garis tepi, sebaliknya jika belum ter-like maka jumlah like

bertambah dan ikon berubah menjadi hati berwarna merah. Saat program dijalankan, tampilan aplikasi menampilkan ikon hati besar bersama teks jumlah like awal yaitu 10, kemudian nilai tersebut berubah mengikuti tindakan pengguna ketika ikon ditekan. Hal ini menunjukkan bahwa mekanisme pengelolaan state berjalan secara responsif dan dapat menyesuaikan tampilan dengan perubahan data secara real-time.

Output:



3. Tugas

Source code:

```

import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:simple_circular_progress_bar/simple_circular_progress_bar.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatefulWidget {
  const MyApp({super.key});

  @override
  State<MyApp> createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
  late ValueNotifier<double> _valueNotifier;
  late double counter;

  @override
  void initState() {
    super.initState();
    _valueNotifier = ValueNotifier(0.0);
    counter = 0.0;
  }

  @override
  void dispose() {
    _valueNotifier.dispose();
    super.dispose();
  }

  void incrementCounter() {
    setState(() {
      if (counter < 33) {
        counter++;
        _valueNotifier.value = (counter / 33) * 100;
      }
    });
  }
}

const SizedBox(height: 50),
ClipRect(
  borderRadius: BorderRadius.circular(50),
  child: InkWell(
    onTap: incrementCounter,
    child: Container(
      decoration: const BoxDecoration(color: Colors.white),
      child: const Icon(Icons.fingerprint, size: 125),
    ), Container
  ), InkWell
), ClipRect
), Column
), Center
), SafeArea
floatingActionButton: FloatingActionButton(
  onPressed: resetCounter,
  child: const Icon(Icons.refresh_outlined),
), FloatingActionButton
), Scaffold
); MaterialApp
}
}

void resetCounter() {
  setState(() {
    counter = 0.0;
    _valueNotifier.value = (counter / 33) * 100;
  });
}

@override
Widget build(BuildContext context) {
  SystemChrome.setSystemUIOverlayStyle(
    const SystemUIOverlayStyle(statusBarColor: Colors.transparent));
}

return MaterialApp(
  debugShowCheckedModeBanner: false,
  theme: ThemeData(
    colorScheme: ColorScheme.fromSeed(seedColor: Colors.green),
    useMaterial3: true,
  ), ThemeData
  home: Scaffold(
    backgroundColor: Colors.green,
    body: SafeArea(
      child: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text('${counter.round()}', style: const TextStyle(fontSize: 50)),
            SimpleCircularProgressbar(
              progressColors: [Colors.amberAccent],
              size: 300,
              progressStrokeWidth: 20,
              backStrokeWidth: 10,
              mergeMode: true,
              maxValue: 100,
              animationDuration: 0,
              valueNotifier: _valueNotifier,
              onDetText: (value) {
                return Text(
                  '${(value.toInt() / 3).round()}',
                  style: const TextStyle(fontSize: 170),
                ); Text
              }, Text
            ), SimpleCircularProgressbar
          ],
        ),
      ),
    ),
  ),
)

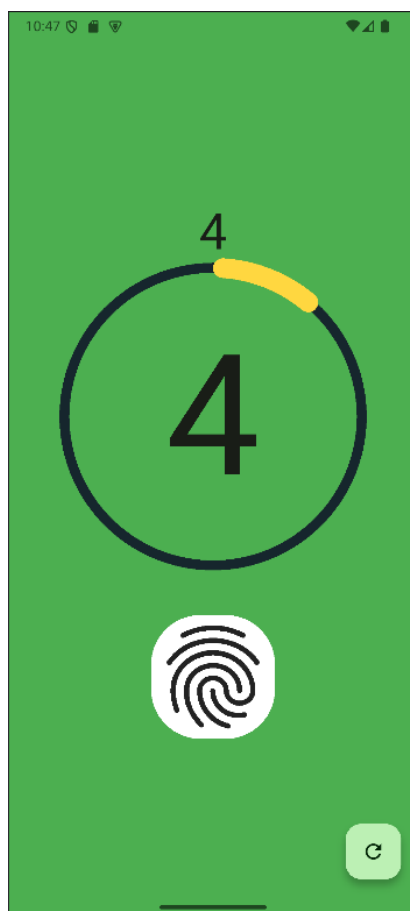
```

Penjelasan kode program

Program tasbih digital ini dibuat menggunakan Flutter dengan memanfaatkan StatefulWidget dan ValueNotifier sebagai mekanisme pengelolaan state yang memungkinkan tampilan berubah secara dinamis selama aplikasi berjalan. Variabel counter berfungsi sebagai penyimpan jumlah hitungan tasbih, sedangkan _valueNotifier digunakan untuk mengatur presentase progres pada SimpleCircularProgressBar. Ketika

pengguna menekan tombol fingerprint, fungsi `incrementCounter()` akan dijalankan dan melalui `setState()` nilai counter bertambah hingga batas maksimal 33 hitungan, sementara nilai progres bar juga meningkat mengikuti perubahan data. Sebaliknya, tombol reset yang berada pada `FloatingActionButton` mereset hitungan ke angka awal dan memperbarui progres kembali ke 0 sehingga pengguna dapat memulai hitungan dari awal kapan saja.

Ouput:



IV. Kesimpulan

Berdasarkan hasil percobaan yang telah dilakukan, dapat disimpulkan bahwa penggunaan `StatefulWidget` merupakan komponen yang penting dalam pengembangan aplikasi Flutter ketika aplikasi memerlukan perubahan tampilan secara dinamis. Melalui penerapan fungsi `setState()`, perubahan nilai atau data yang terjadi selama proses interaksi pengguna dapat ditampilkan secara langsung tanpa perlu memuat ulang keseluruhan aplikasi. Implementasi pada percobaan Counter App dan Like Button menunjukkan bahwa state dapat memengaruhi UI secara real-time, sementara pada percobaan Tasbih Digital, penggunaan `ValueNotifier` dan `SimpleCircularProgressBar` memberikan pemahaman tambahan mengenai visualisasi progres pada aplikasi. Dengan demikian, praktikum ini memberikan pengalaman nyata dalam pengelolaan state pada Flutter dan menjadi dasar penting untuk mempelajari manajemen state yang lebih kompleks di tingkat pengembangan aplikasi berikutnya.